



AIA – RNCP38777

Joel Gloaguen

22 septembre 2026



Agenda

- Bloc 1 — Gouvernance des données Spotify
- Bloc 2 — Architecture de données — Affaire Stripe
- Bloc 3 — Détection automatique de fraude
- Bloc 4 — Projet final — Plateforme MLOps



Gouvernance des données Spotify

Bloc 1 — Concevoir et piloter la gouvernance des données





Sommaire

- Spotify en chiffres : l'échelle qui impose la gouvernance
- Trois enjeux critiques
- Maturité data : niveau 3 sur l'échelle CMMI
- Un Centre d'Excellence hybride
- Qui fait quoi sur les données
- Triple conformité : RGPD, CCPA, PCI-DSS
- Stack technologique de gouvernance
- Roadmap : 18 mois, 4 phases
- Risques et mitigations
- ROI : un investissement qui s'impose
- Conclusion



Spotify en chiffres : l'échelle qui impose la gouvernance

678M

utilisateurs actifs

**500
To/jour**

données
comportementales

184

pays, 3 régimes légaux

~16 Md€

revenus annuels

• 678M utilisateurs actifs

• 500 To de données/jour

• 184 pays, 3 régimes légaux

• ~16 Md€ de revenus annuels

Une entreprise de données qui distribue de la musique



Trois enjeux critiques

Légal

Amende RGPD jusqu'à 640M€

4% du CA mondial

Qualité

Précision royalties à 0,1%

Litiges avec les labels

Business

Personnalisation = rétention

Dégradation si données pauvres

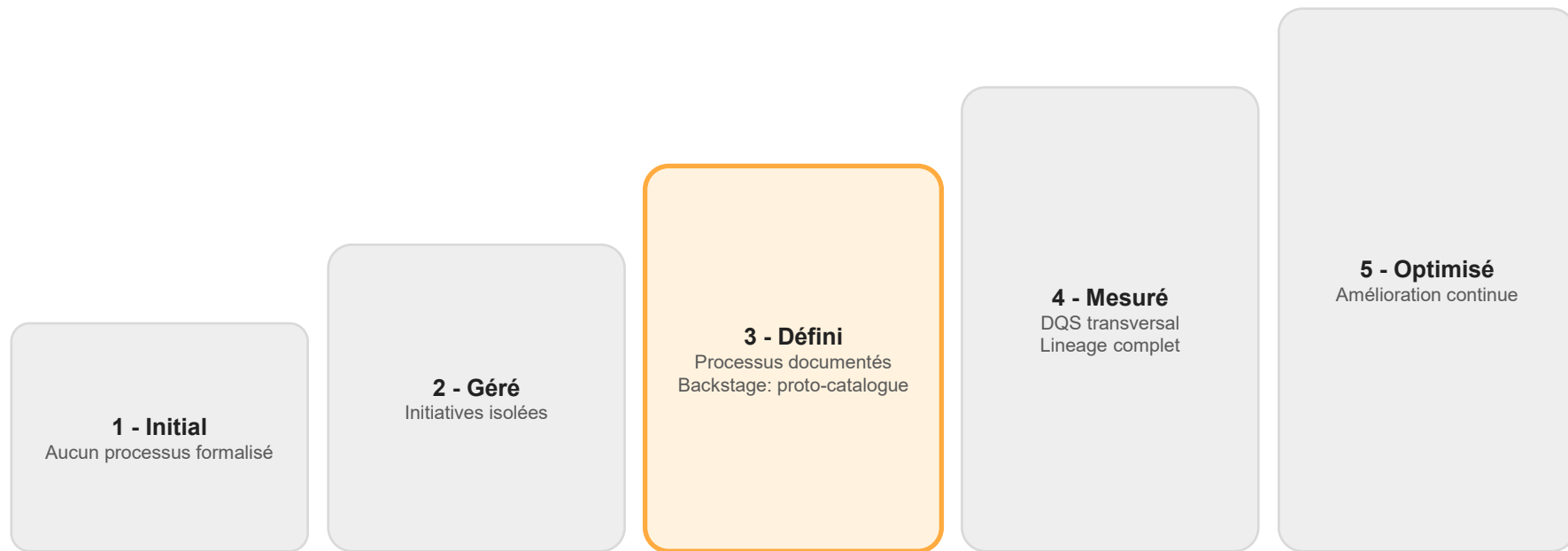
• Légal : amende RGPD jusqu'à 640M€

• Qualité : royalties précises à 0,1%

• Business : recommandation = rétention

Une gouvernance défaillante n'est pas un risque théorique

Maturité data : niveau 3 sur l'échelle CMMI



• Niveau 3 'Défini' atteint

• Backstage = proto-catalogue
data

• Gaps : lineage, silos, formation • Cible : niveau 4 'Mesuré'

Base solide, gaps critiques vers le niveau 4

Un Centre d'Excellence hybride

**Comité de Gouvernance (CDO,
DPO, CISO, CFO)**



CoE Hybride (10 experts)

- Comité : décide et arbitre

- CoE : standards, outils, conformité

- 7 domaines : Owner + Steward

- Respecte le modèle Tribes/Squads

Standards centraux, exécution décentralisée

Qui fait quoi sur les données

Activité	Owner	Steward	DPO	Custodian	Consumer
Règles d'accès	A	R	C	I	I
Qualité des données	A	R	I	I	I
Droits RGPD	C	R	A	C	I
Chiffrement / Sécurité	I	C	C	R	I
Utilisation des données	I	I	I	I	A

• Owner : décide des règles

• Steward : exécute au quotidien

• DPO : valide la conformité

• Custodian : protège techniquement

Plus aucune donnée orpheline

Triple conformité : RGPD, CCPA, PCI-DSS

**RGPD (600M
Européens)**

DPO nommé

Registre Art.30 partiel

Lineage à renforcer

**CCPA (50M
Californiens)**

Opt-out vente en place

Droit de suppression

Signal GPC à renforcer

**PCI-DSS (268M
Premium)**

Tokenisation des paiements

TLS 1.3

Audit QSA à planifier

• Délai de réponse unique : 30 jours

• Opt-in marketing pour tous

• Paiements tokenisés (Stripe)

• Droit à l'oubli vs modèles ML : cas à part

Stratégie : la norme la plus contraignante pour tous

Stack technologique de gouvernance

Catalogue

Apache Atlas

Qualité

Great Expectations

dbt tests

Sécurité

HashiCorp Vault

Streaming

Confluent

Schema Registry

Accès

GCP IAM

BigQuery Column/Row Security

Monitoring

GCP DLP

Grafana

Splunk

• Atlas : catalogue + lineage

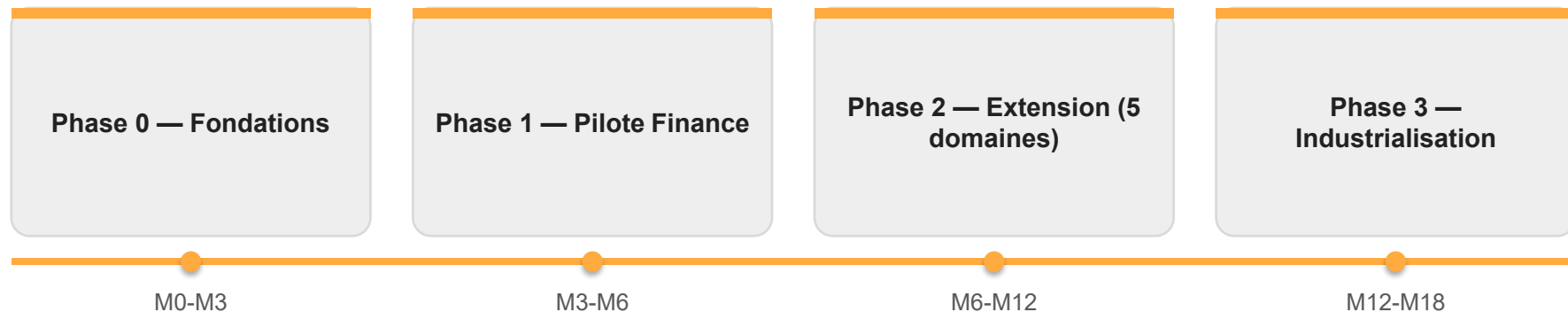
• Great Expectations : DQS automatisé

• Vault : rotation des secrets

• DLP : détection PII automatique

Priorité à l'open-source, intégré à l'existant GCP

Roadmap : 18 mois, 4 phases



• M6 : DQS Finance > 88%

• M12 : 60% du catalogue couvert

• M18 : niveau 4 CMMI confirmé

• Certification PCI-DSS renouvelée

Finance en premier : ROI démontrable, sponsor motivé

Risques et mitigations

Risque	Mitigation clé
Violation de données RGPD	DLP + chiffrement AES-256, plan de réponse 72h testé
Erreurs de royalties	Great Expectations P0 + réconciliation automatique
Résistance culturelle	Modèle hybride + champions + OKRs qualité
Dérive des données ML	Feature drift monitoring + versioning des datasets
Non-conformité AI Act	Analyse de classification + AI Compliance Officer

- Aucun risque en zone critique

- Tokenisation = risque réduit par conception

- AI Act : risque émergent à anticiper

Aucun risque en zone critique : Privacy by Design

ROI : un investissement qui s'impose

~6M€

investissement sur 18
mois

- CoE hybride adapté à la culture

576M€

amende RGPD évitée

- Conformité RGPD/CCPA/PCI-DSS unifiée

45M€

litiges royalties évités

- Roadmap mesurable sur 18 mois

>1000%

ROI à 3 ans

- Prérequis à l'architecture Data Mesh

Niveau 3 'Défini' → niveau 4 'Mesuré'



Conclusion





Questions — Bloc 1 ?

Architecture de données — Affaire Stripe

Bloc 2 — Concevoir et déployer des architectures de données (pour l'IA)



Sommaire

- Contexte — Stripe, une infrastructure de paiement critique
- Architecture globale — 3 couches complémentaires
- Modèle OLTP — 8 entités en 3NF (notation Merise)
- Modèle OLAP — schéma en étoile
- Modèle NoSQL — DynamoDB Single Table Design
- Pipeline batch — CDC sans impact sur l'OLTP
- Pipeline temps réel — scoring de fraude en < 30 ms
- Sécurité et conformité par construction
- Intégration ML — cycle de vie de la détection de fraude
- Bilan en production — SLA atteints
- Conclusion — fiabilité, performance, conformité

Contexte — Stripe, une infrastructure de paiement critique

**>1 000
Mds \$**

Transactions traitées / an

• Fiabilité financière

3 000 / s

TPS nominal (pic 50
000/s)

• Détection de fraude

99.99 %

Disponibilité cible

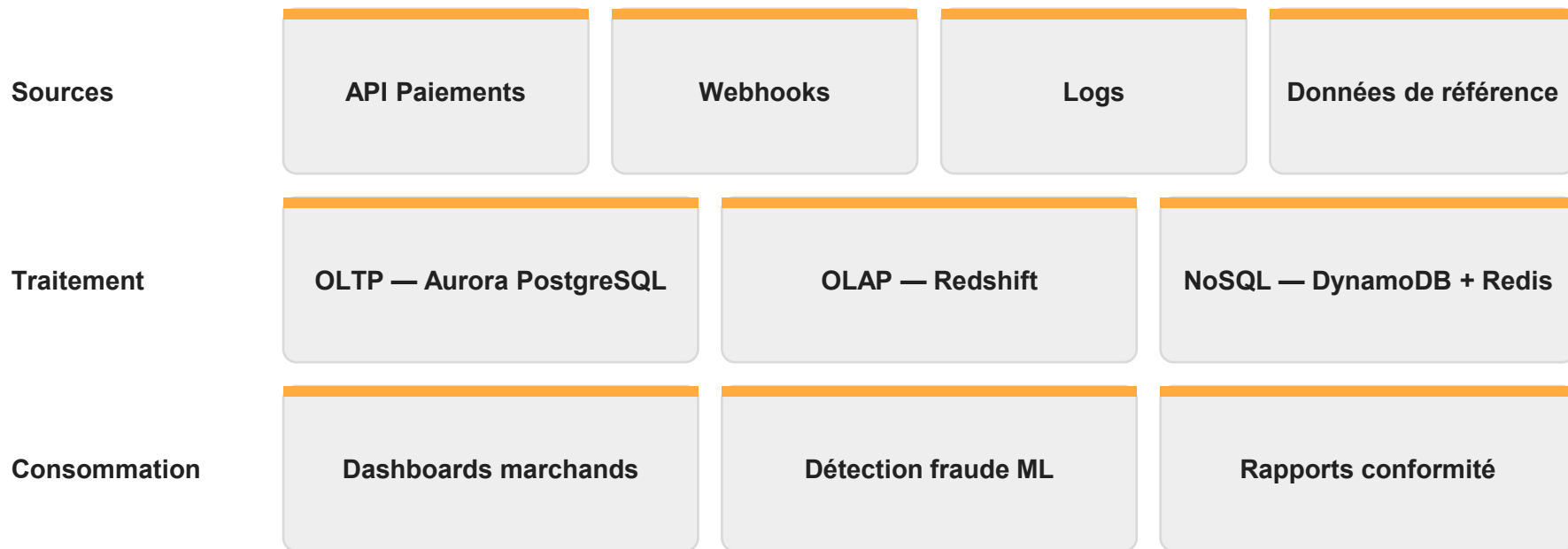
• Conformité PCI-DSS / RGPD

< 100 ms

Latence autorisation

Fintech paiements à l'échelle mondiale

Architecture globale — 3 couches complémentaires

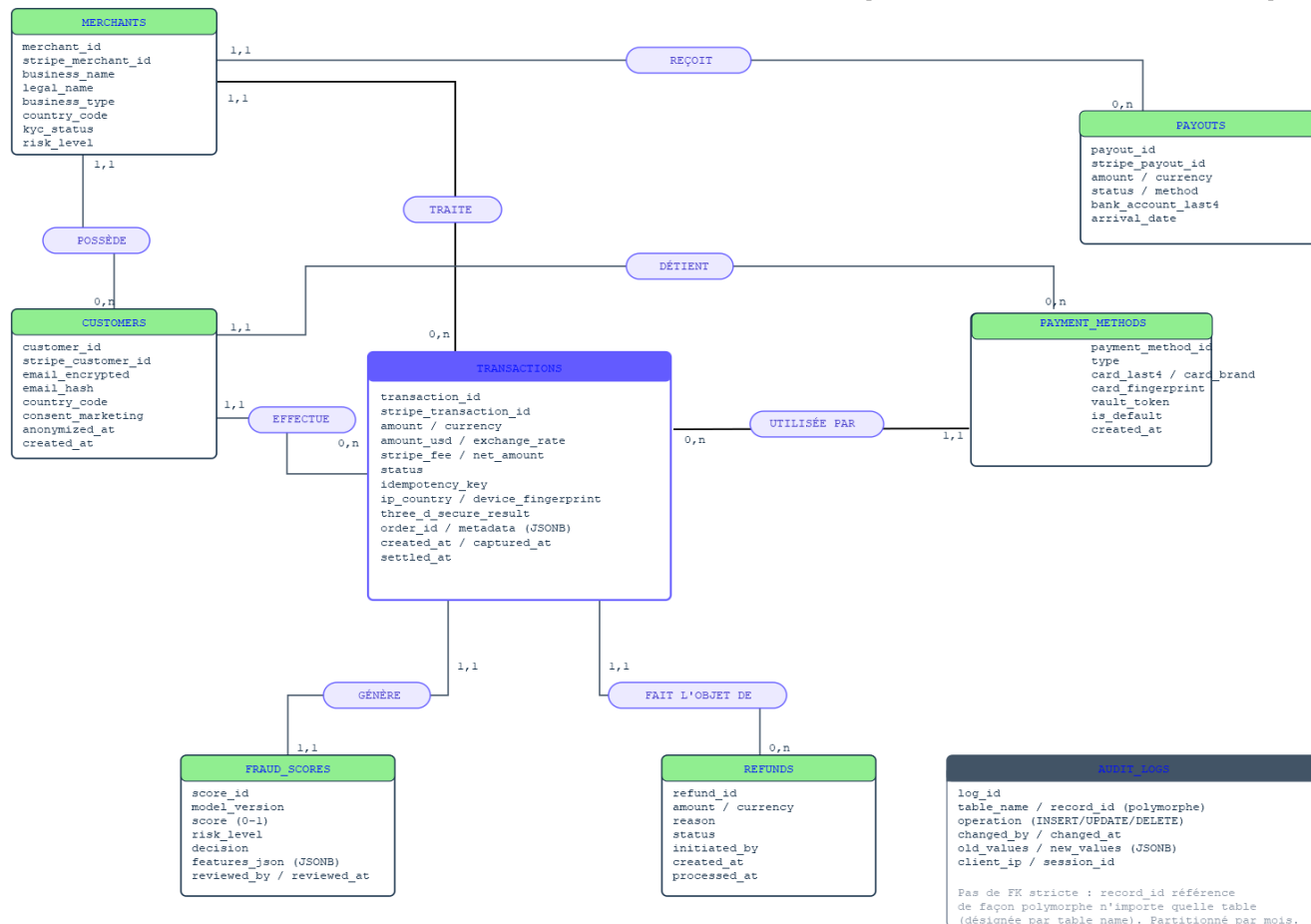


• 3 systèmes, 3 rôles

• La bonne techno au bon endroit

• Synchronisation par CDC

Modèle OLTP — 8 entités en 3NF (notation Merise)



• 3NF stricte + contraintes CHECK

• TRANSACTIONS = entité pivot

• Partitionnement par date (>100M lignes)

Modèle OLAP — schéma en étoile

create materialized view mv_daily_revenue_by_merchant as

select

dt.full_date,

dt.year,

dt.month,

dm.merchant_key,

dm.stripe_merchant_id,

dm.business_name,

dm.country_code,

dm.size_segment,

COUNT(ft.transaction_key) as transaction_count,

SUM(ft.amount_usd) as gross_revenue_usd,

SUM(ft.stripe_fee_usd) as total_fees_usd,

SUM(ft.net_amount_usd) as net_revenue_usd,

SUM(case when ft.is_succeeded then 1 else 0 end) as succeeded_count,

SUM(case when ft.is_fraud then 1 else 0 end) as fraud_count,

ROUND(SUM(case when ft.is_succeeded then 1.0 else 0 end)

/ nullif(COUNT(*), 0) * 100, 2) as success_rate_pct

from

fact_transactions ft

join dim_time dt on

ft.time_key = dt.time_key

join dim_merchant dm on

ft.merchant_key = dm.merchant_key

where

dm.is_current = true

group by

dt.full_date,

dt.year,

dt.month,

dm.merchant_key,

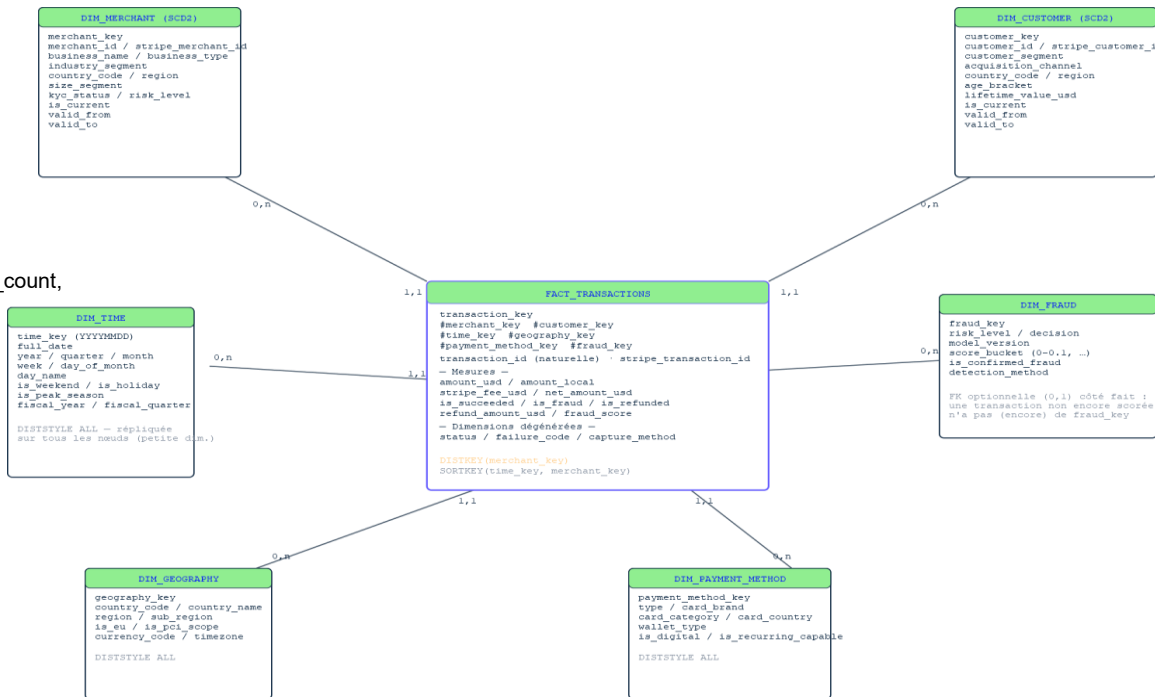
dm.stripe_merchant_id,

dm.business_name,

dm.country_code,

dm.size_segment

with no schema BINDING;



Modèle NoSQL — DynamoDB Single Table Design

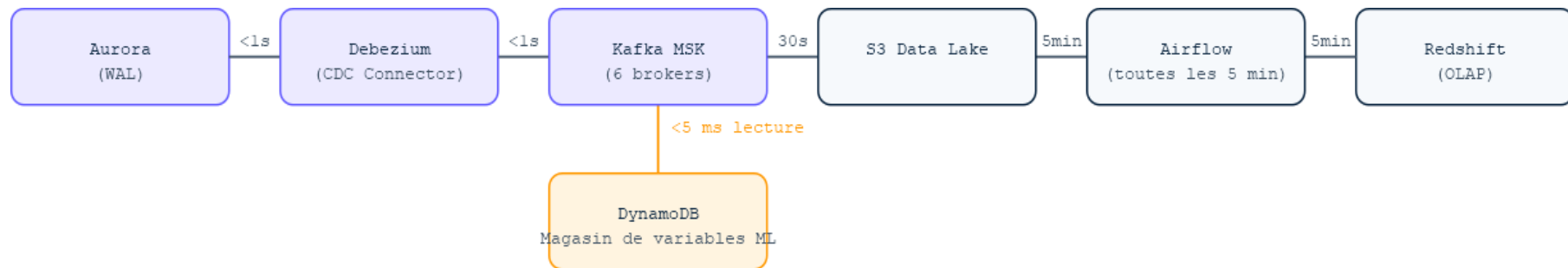
Clé de partition (PK)	Clé de tri (SK)	Contenu
CUSTOMER#id	FEATURE#fraud#timestamp	Features de fraude client
TRANSACTION#id	SCORE#realtime	Score de fraude temps réel
SESSION#id	METADATA	Session utilisateur

- 1 table, préfixes par type d'entité

- 3 GSI (marchand, modèle, risque)

- Cache DAX < 1 ms

Pipeline



Sécurité et conformité par construction

Sécurité

Tokenisation PAN

AES-256 + TLS 1.3

VPC Zero Trust (IAM,
MFA)

Audit → CloudTrail →
SIEM

Conformité

PCI-DSS Niveau 1

RGPD (TTL, droit à l'oubli)

CCPA (localisation données)

Scalabilité

Aurora : 15 réplicas

Redshift : 4 → 32 nœuds

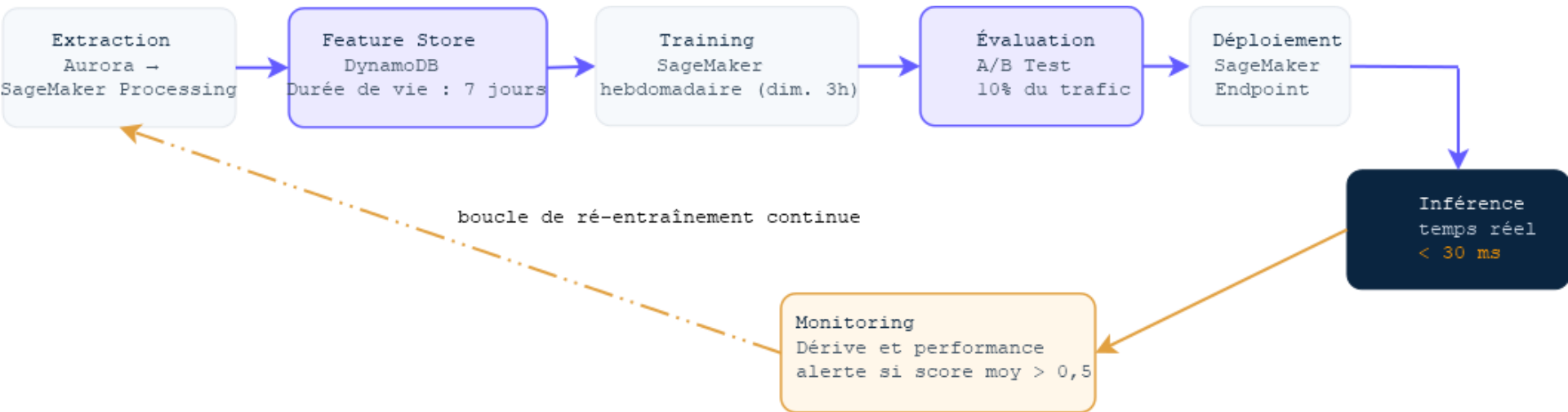
DynamoDB : auto-scaling

• Aucun numéro de carte stocké

• Chiffrement au niveau du champ

• Montée en charge automatisée

Intégration ML — cycle de vie de la détection de fraude

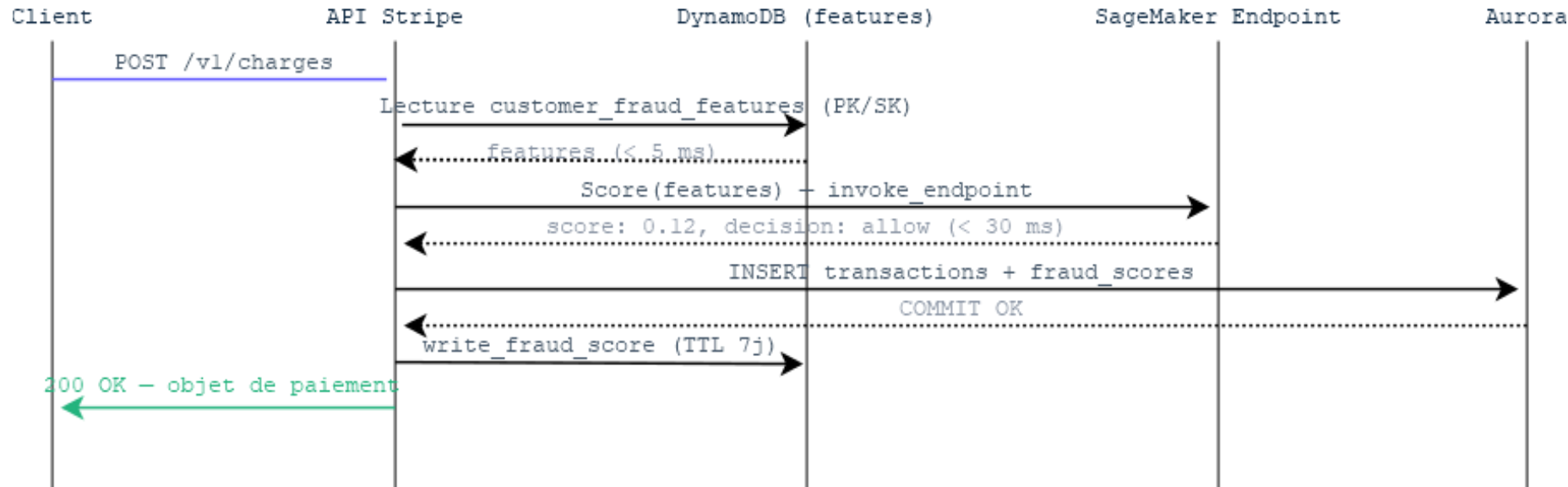


- Chaque score versionné

- Explicabilité (SHAP) stockée

- Alerte si score moyen > 0,5

Intégration ML — inférence temps réel



Bilan en production — SLA atteints

SLA	Valeur mesurée
Disponibilité OLTP	99.99 % (< 52 min/an)
Latence autorisation	< 100 ms (p99)
Failover Aurora	< 30 s (22 s mesuré)
Fraîcheur OLAP	< 5 min (CDC continu)
RTO / RPO	< 1 h / < 1 s

- GameDay mensuel

- Infrastructure as Code (Terraform)

- Blue-Green Deployment sans coupure

Conclusion — fiabilité, performance, conformité

- 3 systèmes, 1 pipeline CDC
- Conformité par construction
- Capacité à évoluer préservée



Conclusion — Bloc 2



Questions — Bloc 2 ?

Détection automatique de fraude

Bloc 3 — Concevoir et mettre en œuvre des pipelines de données (pour l'IA)



Sommaire

- 1 milliard d'euros de fraude par an — un besoin double
- Architecture retenue : simple et assumée
- Flux temps réel — de la transaction à l'alerte
- Architecture globale — deux boucles, un point de croisement
- Réentraînement — un push de code, pas un DAG nocturne
- Modèle ML — un entraînement réel, lu honnêtement
- Sécurité & conformité — ce qui reste à corriger
- Qualité des données & monitoring — honnêtes sur les limites
- Trois itérations : de jefraudai à FraudHunter
- Stack technique réellement utilisée
- Bilan et ce qui reste à faire

1 milliard d'euros de fraude par an — un besoin double

1 Md €

fraude carte UE/an (BCE 2019)

- Alerte immédiate si fraude

2

besoins : alerte immédiate + revue
quotidienne

- Revue quotidienne de la veille

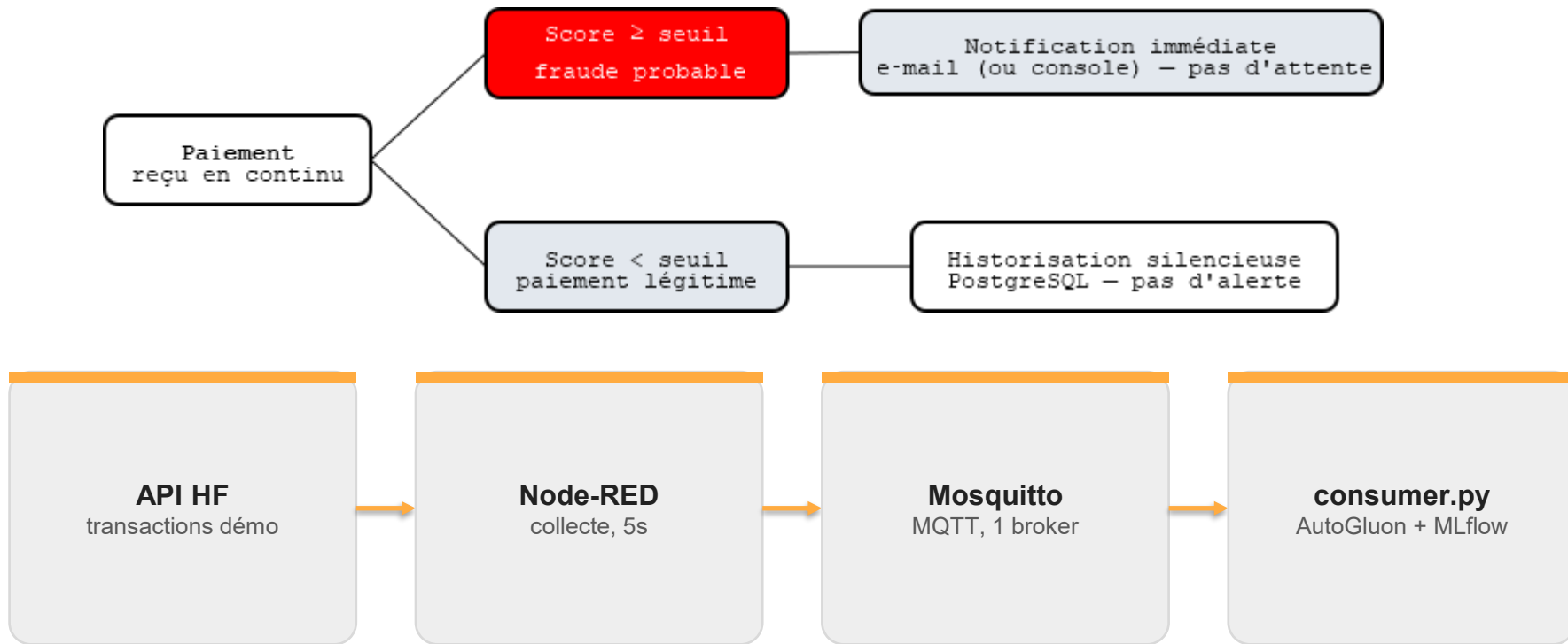
0

service tiers payant utilisé

- Pipeline évalué, pas l'algo

Le cahier des charges évalue le pipeline de données, pas la performance de l'algorithme

Besoin et Architecture retenue : simple et assumée



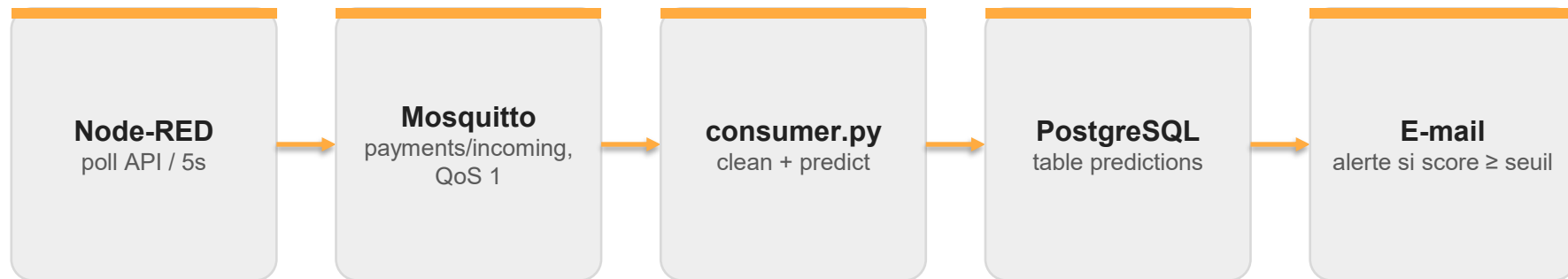
• Cahier des charges autorise l'écart

• Zéro service tiers payant

• 100% reproductible en local

Pas la Lambda Architecture envisagée initialement (Kafka / Flink / Airflow)

Flux temps réel — de la transaction à l'alerte

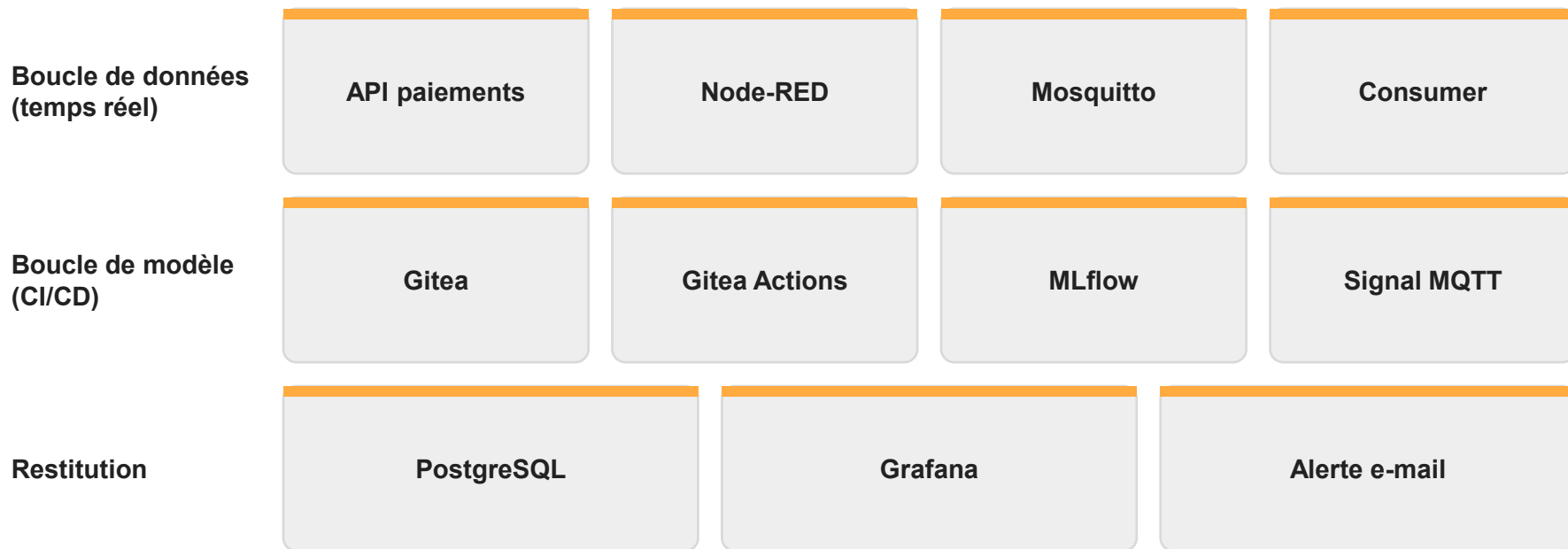


- Sans modèle chargé → transaction ignorée

- Pas de budget de latence formalisé

- Repli console si SMTP absent

Architecture globale — deux boucles, un point de croisement



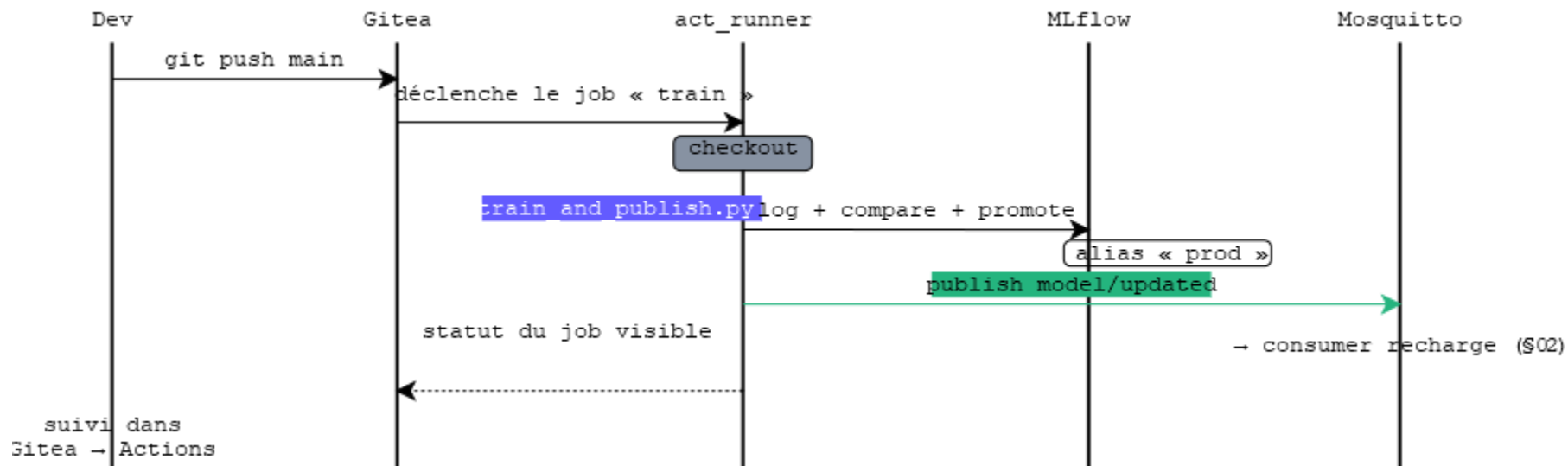
• 1 seul broker Mosquitto, 2 topics

• Gitea Actions publie lui-même le signal

• Consumer ne recharge que sur signal reçu

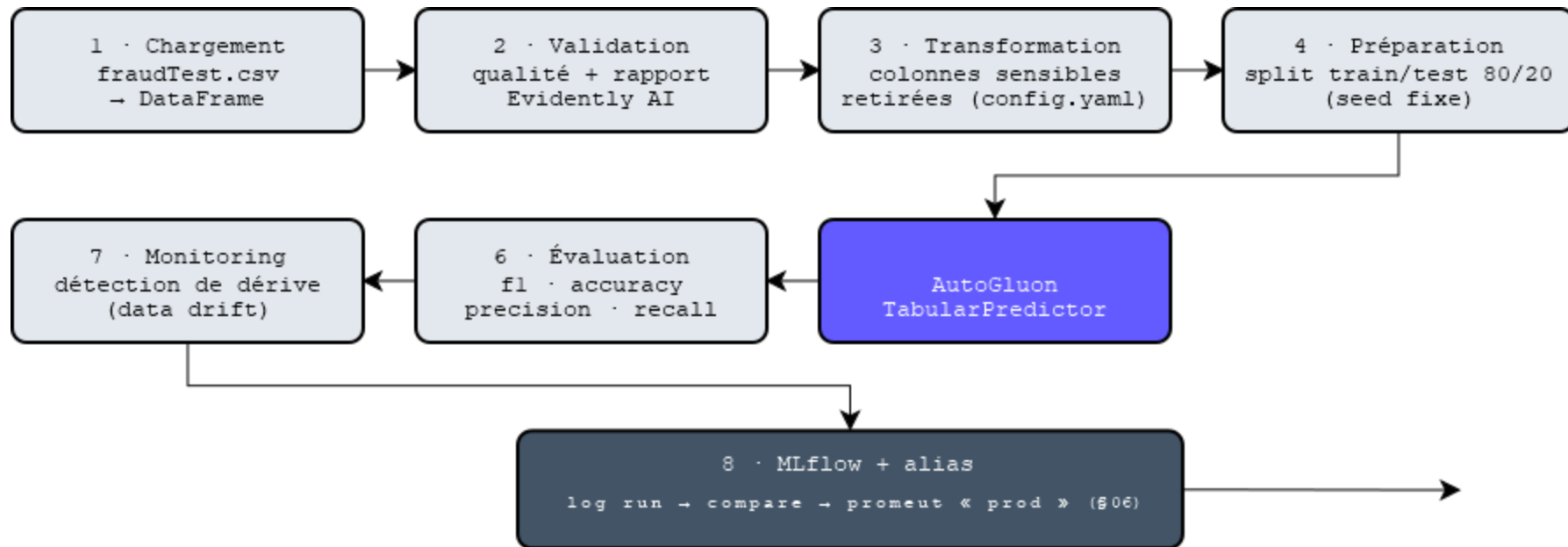
Le consumer Python est le seul point où les deux boucles se croisent

Pipeline — CI/CD



Déclencheur réel : git push sur main (Gitea Actions)

Réentraînement — un push de code, pas de DAG



- Promotion seulement si meilleur (F1 d'abord)
- Consumer recharge sans redémarrage
- Signal MQTT publié dans tous les cas

Déclencheur réel : git push sur main (Gitea Actions)

Sécurité & conformité — ce qui reste à corriger

En place

Secrets en variables d'environnement
Réseau Docker isolé (fraudhunter-net)
Accès au socket Docker limité au runner CI

À corriger avant tout usage réel

cc_num stocké en clair dans predictions
Pseudonymisation conçue, non codée
Pas de chiffrement au repos configuré

- Dataset Kaggle public (fraudTest.csv)
- Pas de données clients réelles utilisées
- Pseudonymisation = priorité avant mise en réel

Qualité des données & monitoring — honnêtes sur les limites

Qualité des données	Validation nulls / doublons	Rapport Evidently	Non bloquant
Monitoring technique	Grafana	Lecture directe PostgreSQL	Pas de Prometheus
Monitoring modèle	Dérive calculée à l'entraînement		Pas de suivi continu en production

- Validation ne bloque jamais le pipeline

- Pas d'alerte automatique sur dérive

- Feature store absent de cette version

Trois itérations : de jefraudai à FraudHunter



• Kafka/cloud → Mosquitto/Node-RED → CI/CD locale

• Chaque itération = un choix assumé et documenté

• Vision cible non reniée, sciemment simplifiée

Les livrables 01 à 07 documentent la version FraudHunter, retenue et présentée aujourd'hui

Stack technique réellement utilisée

Ingestion

Node-RED

Mosquitto (MQTT)

ML

AutoGluon

MLflow

CI/CD

Gitea

Gitea Actions

Stockage & restitution

PostgreSQL

Grafana

Alerting

SMTP

repli console

• Zéro service cloud ou tiers payant

• 100% reproductible en local

• gitea-runner = seul accès au socket
Docker hôte

9 services Docker Compose sur un unique réseau bridge fraudhunter-net

Bilan et ce qui reste à faire

Statut	Élément
En place	Ingestion temps réel, inférence, alerte, CI/CD, promotion de modèle
À vérifier	Dashboard Grafana en conditions réelles, chaîne CI de bout en bout, poids du CSV
Non branché	watchdog.py (présent, jamais démarré), ré-entraînement planifié en cron

- Vidéo de démonstration à enregistrer
- Pseudonymisation cc_num = priorité
- Feature store et dérive continue absents



Conclusion — Bloc 3



Questions — Bloc 3 ?

Projet final — Plateforme MLOps watt_on_the_weather

Bloc 4 — AI Solutions Design, Déploiement et Monitoring



Sommaire

- Démarche & enjeu métier
- Objectifs mesurables
- Données
- Cahier des charges → réalisation
- Architecture globale — 11 services
- Boucle de simulation & détection de dérive
- Réentraînement — deux chemins, un seul code
- Pipeline CI/CD Gitea
- Monitoring Grafana
- Stack technique
- Risques & conformité
- Bilan

Démarche & enjeu métier

Sans prévision

Pic observé
Achat en urgence (marché spot)
Marge dégradée

Avec prévision J+5

Prédiction J pour J+1..J+5
Achat anticipé
5 jours pour ajuster
Coûts réduits

- Cahier des charges strict

- Outils déjà maîtrisés

- Un rôle par service

Prédire pour acheter l'énergie à l'avance plutôt que dans l'urgence

Objectifs mesurables

J+5

Horizon de prédiction

<15%

Seuil MAPE

0

Interruption de service

100%

Autonomie (local)

Données



- Aucune donnée personnelle

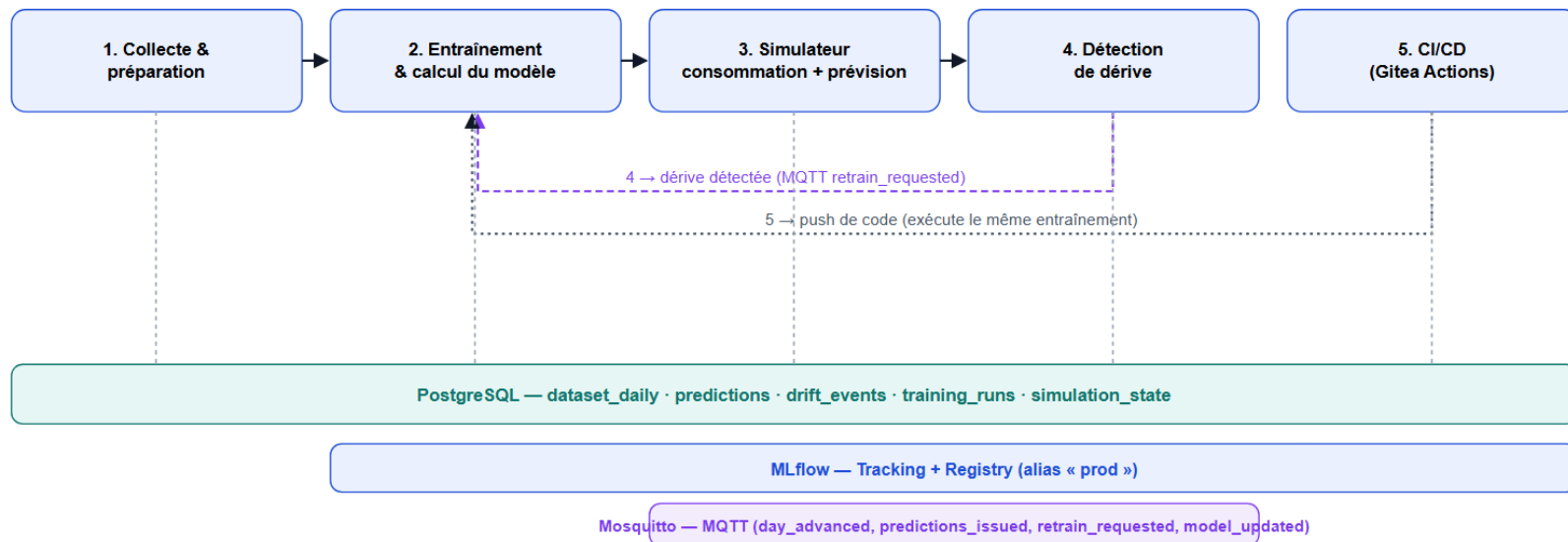
- Bornes calendaires, jamais de mélange aléatoire

Découpage chronologique strict : G1 entraînement (~22 mois) · G2 comparaison (3 mois, jamais vus) · G3 simulation (5 mois, rejoués jour par jour)

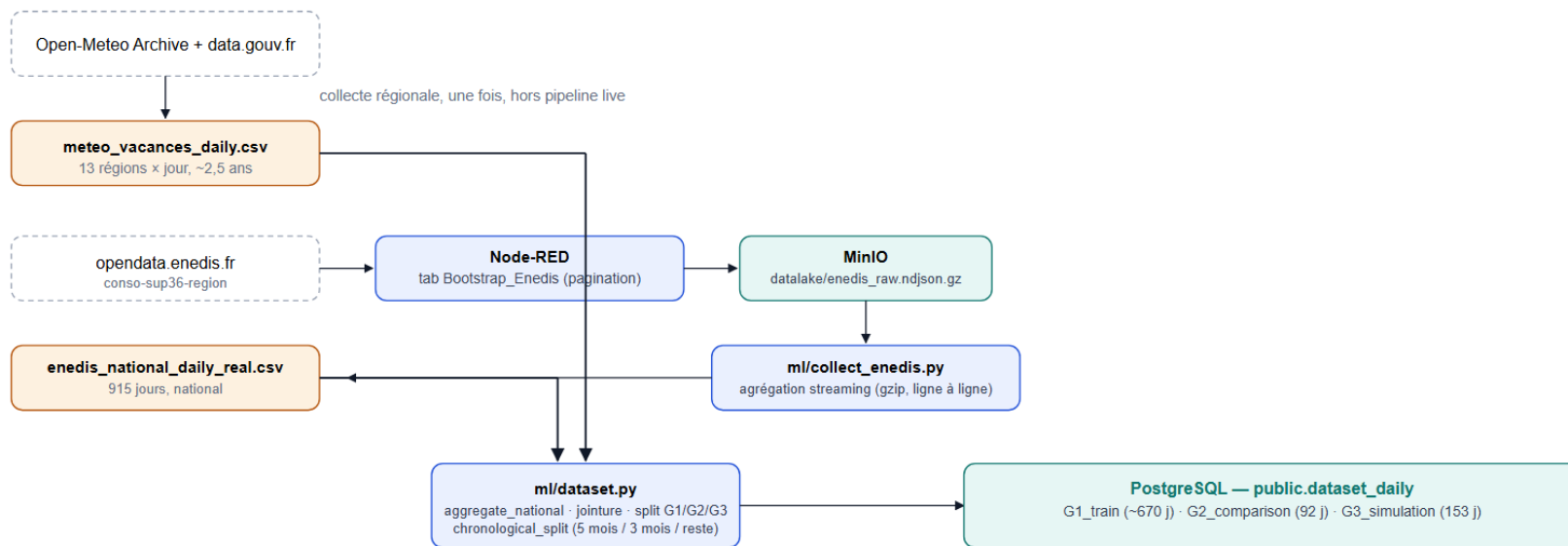
Cahier des charges → réalisation

Exigence	Réalisation
Prédire J+1 à J+5	LightGBM, feature horizon
Flux « temps réel »	POST /simulation/advance
Tableau de bord	Grafana, 3 courbes
Boucle simulation / dérive	run_tick() + Node-RED
Réentraînement non bloquant	MQTT vers service trainer
CI/CD	Gitea Actions
Traçabilité du modèle	MLflow, alias prod
Autonomie	Docker Compose, 100% local

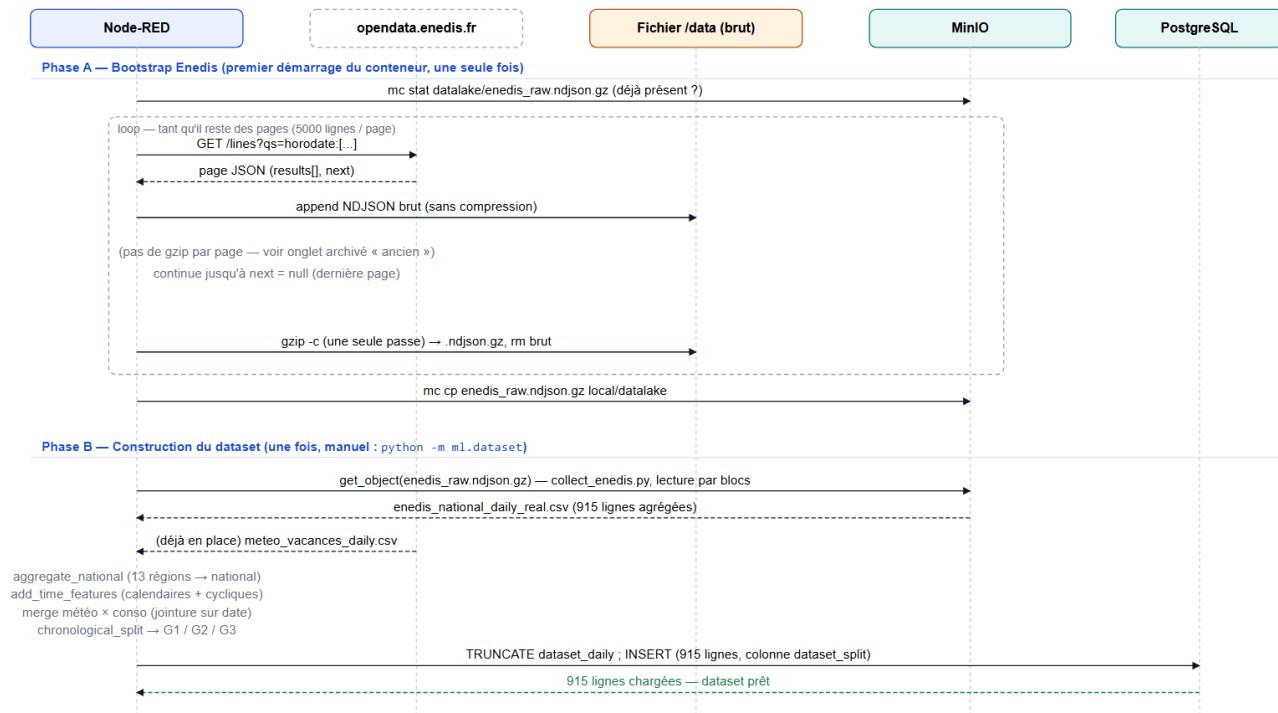
Vue d'ensemble — cinq étapes du pipeline



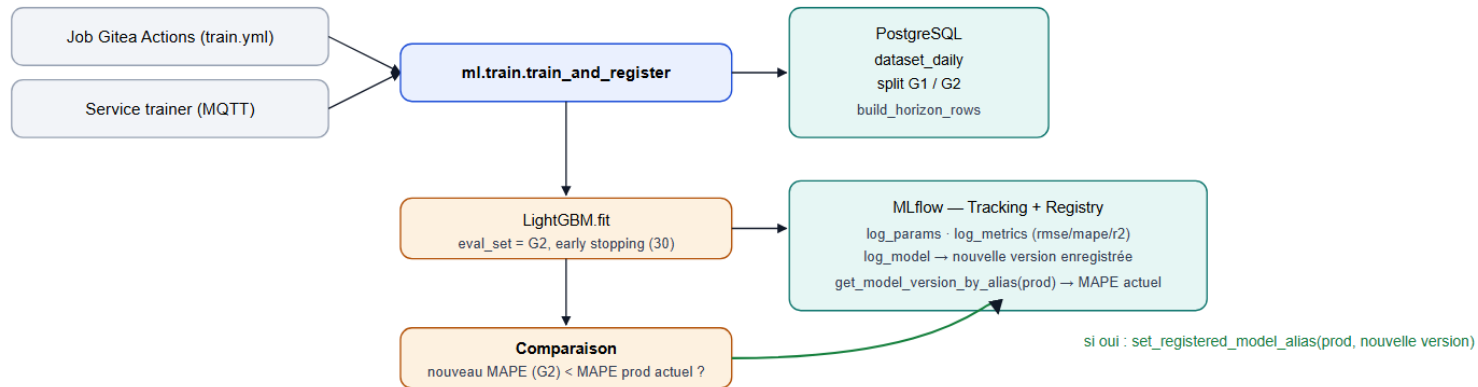
Étape 1 — Collecte & préparation (architecture)



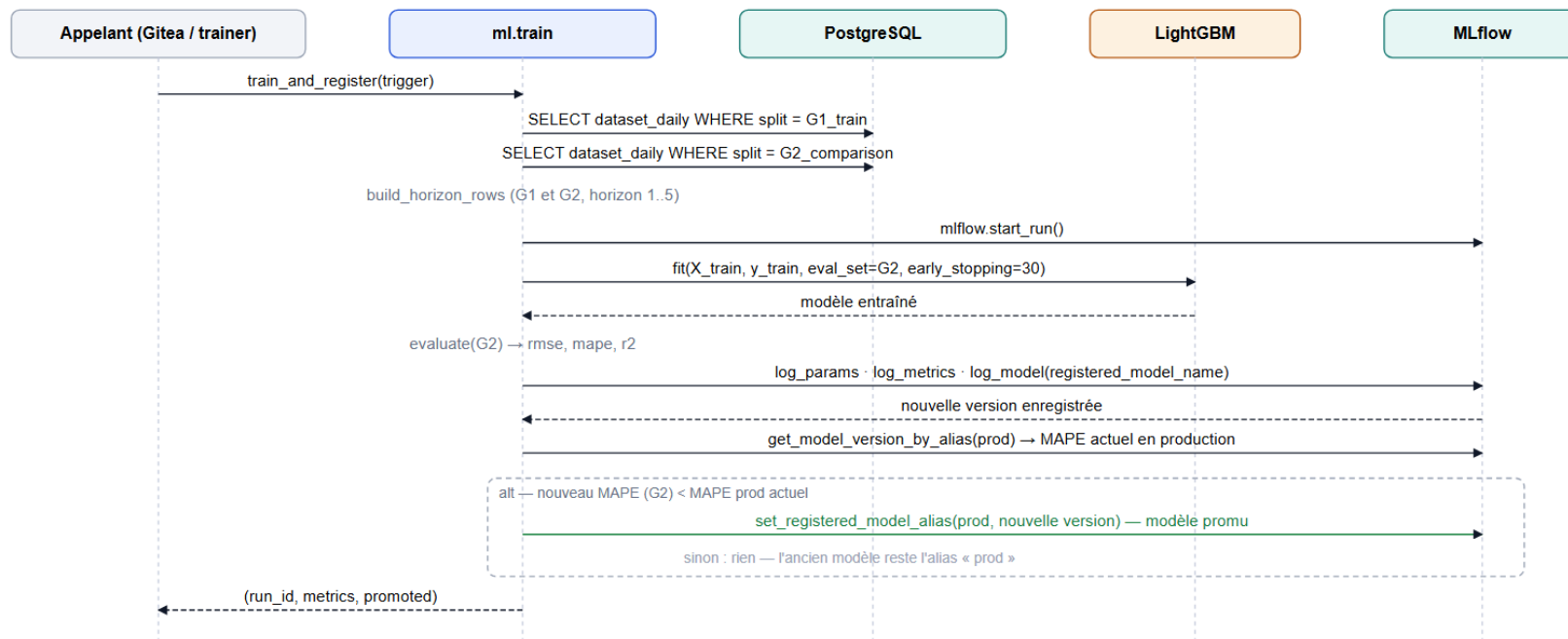
Étape 1 — Collecte & préparation (séquence)



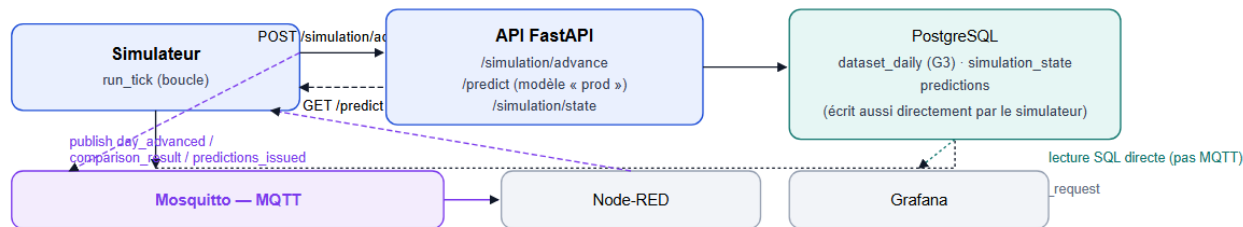
Étape 2 — Entraînement (architecture)



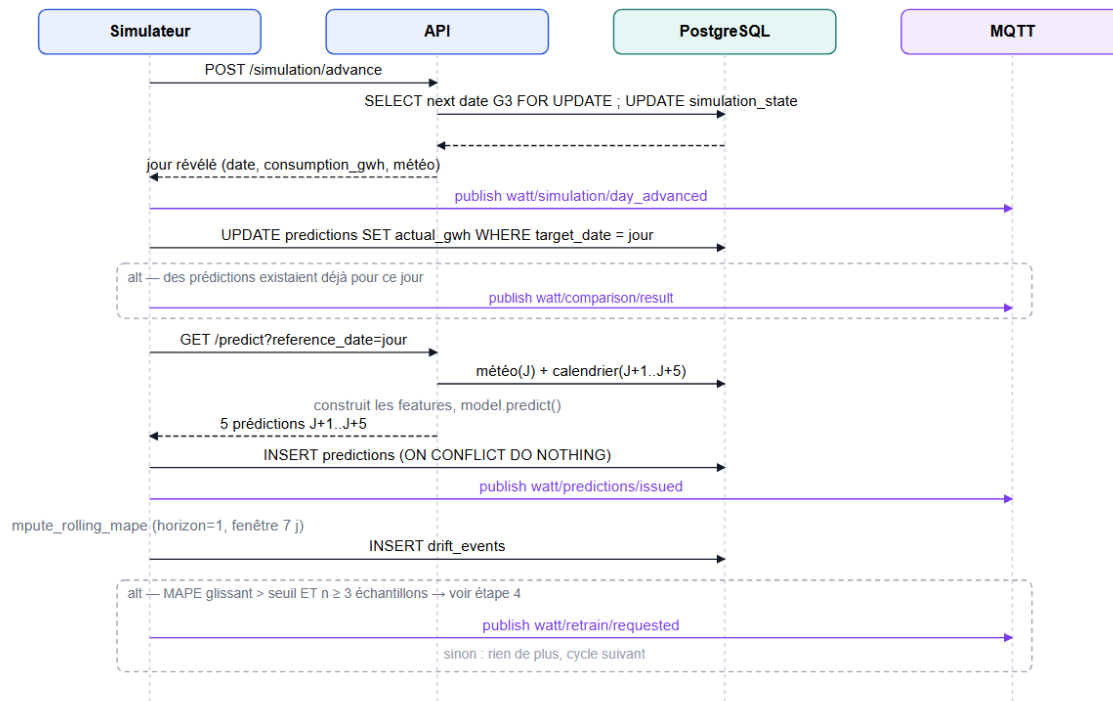
Étape 2 — Entraînement (séquence)



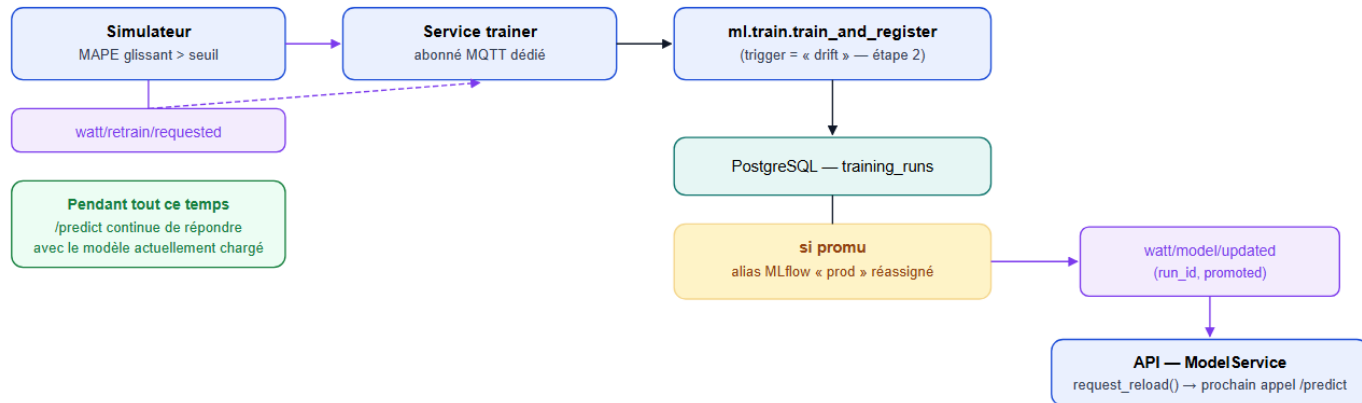
Étape 3 — Simulateur (architecture)



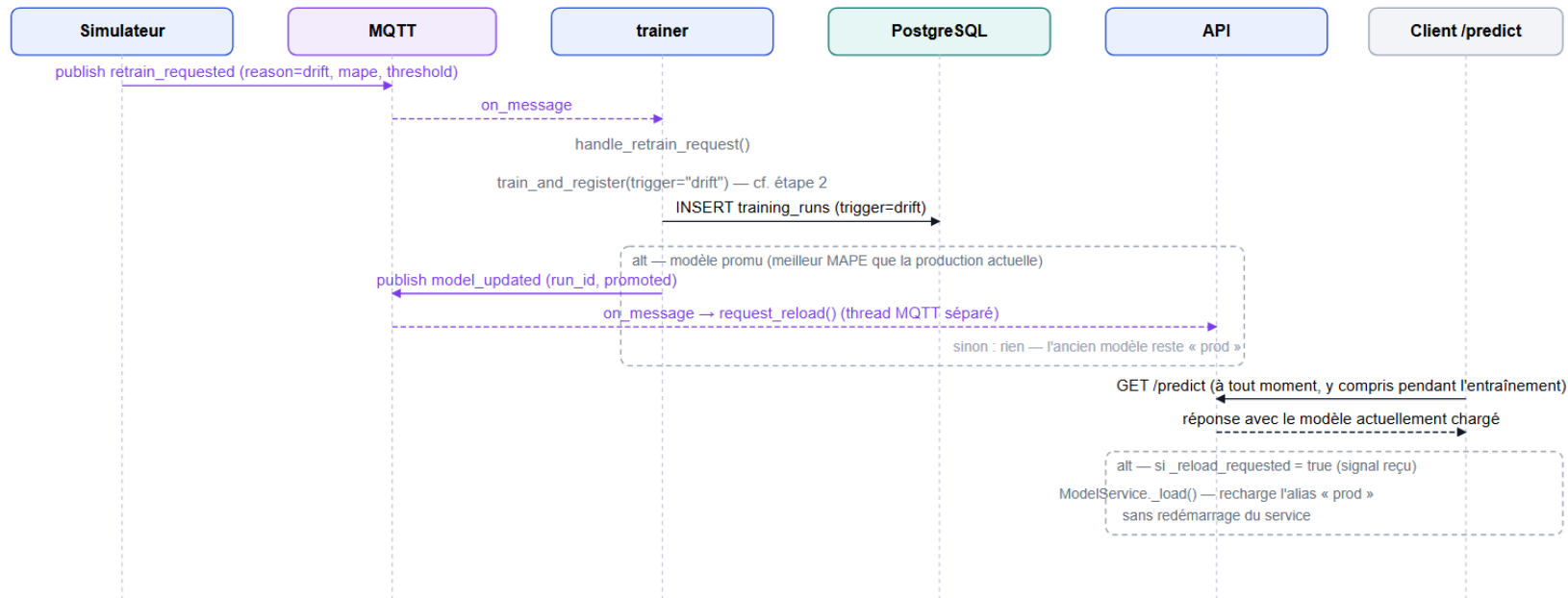
Étape 3 — Simulateur (séquence)



Étape 4 — Détection de dérive (architecture)



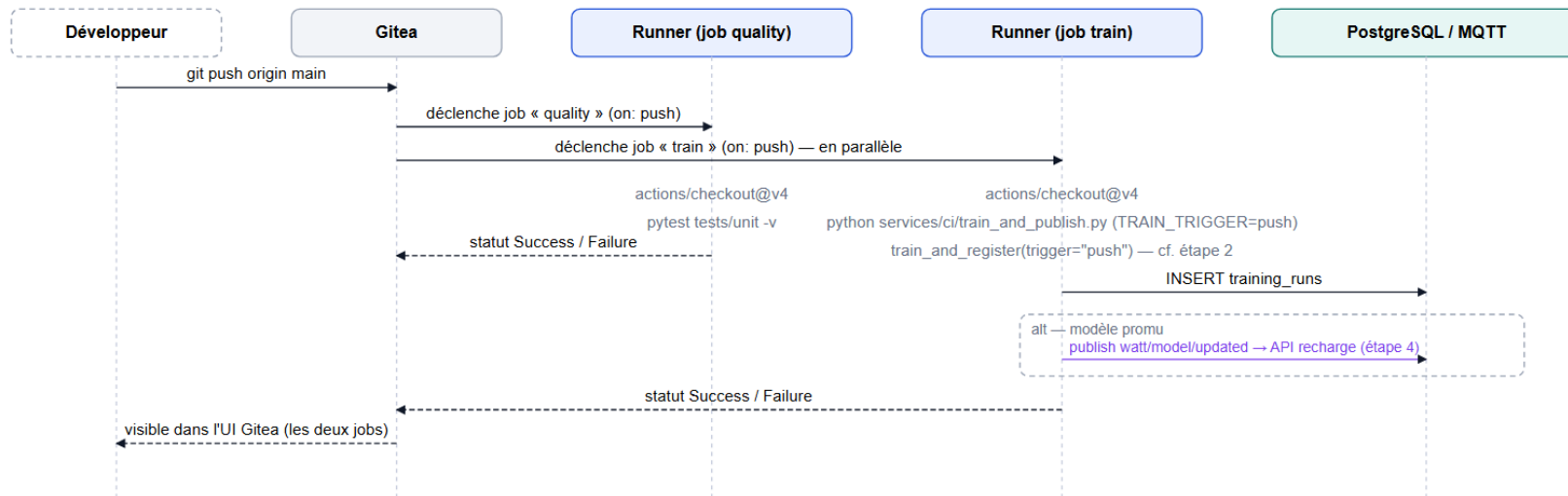
Étape 4 — Détection de dérive (séquence)



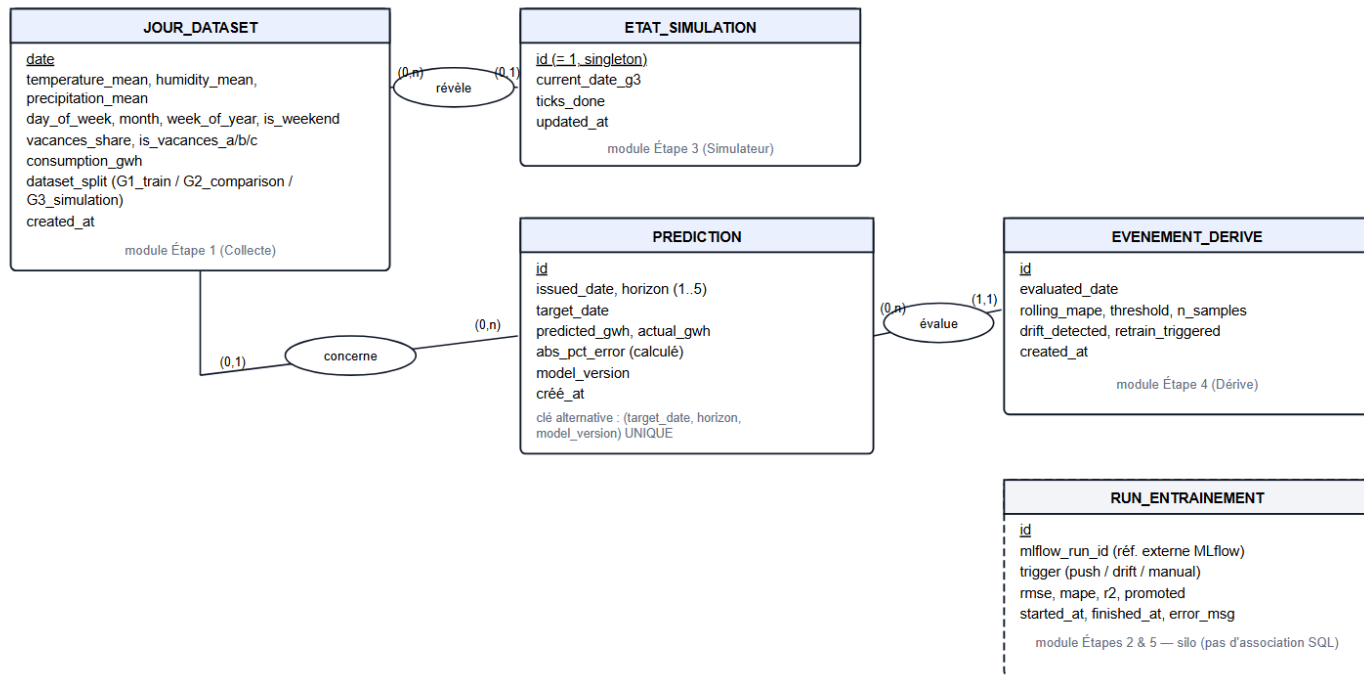
Étape 5 — CI/CD Gitea (architecture)



Étape 5 — CI/CD Gitea (séquence)



Modèle conceptuel de données (Merise)



Correspondance panneaux Grafana

Panneau	Type	Table(s) source	Colonnes clés
Réel vs Prédiction J+5 vs MAPE glissant	timeseries	dataset_daily, predictions, drift_events	consumption_gwh · predicted_gwh (horizon=5) · rolling_mape
Facteurs d'entrée du modèle — météo et vacances	timeseries	dataset_daily	temperature_mean · vacances_share
Historique de dérive et réentraînements déclenchés	table	drift_events	evaluated_date, rolling_mape, threshold, n_samples, drift_detected, retrain_triggered

Stack technique

Orchestration

Node-RED

Docker Compose

Données

PostgreSQL

Mosquitto (MQTT)

ML

LightGBM

MLflow (registry)

Service & CI/CD

FastAPI

Gitea Actions

Observabilité

Grafana

Jupyter

100% local, aucun service tiers payant

Risques & conformité

Risque	Traitement
Dérive du modèle	MAPE glissant surveillé, réentraînement automatique
Contention CPU pendant l'entraînement	Retarde, n'interrompt jamais l'API
Limite Gitea 1.22	Contournée par le service trainer dédié
Météo cible inconnue à J+5	Proxy = météo du jour J, documenté

Données agrégées nationales/régionales — aucune donnée personnelle

Bilan

915

Jours réels en base

8

Jours de simulation
rejoués

3

Exécutions CI/CD
réussies

1

Réentraînement MQTT
observé

- Pipeline exécuté de bout en bout sur données réelles

- CI/CD Gitea en Success

- Non-interruption de /predict vérifiée pendant un entraînement

Limite assumée : hyperparamètres LightGBM fixés à dire d'expert, cadence 100% automatique non testée en continu longue durée



Conclusion — Bloc 4



Questions — Bloc 4 ?